Opcode 처리

개요

Bus-Mapping의 핵심 기능 중 하나는 EVM의 각 opcode를 처리하여 `ExecStep` 과 관련 연산들을 생성하는 것입니다. 이 문서에서는 opcode별 처리 로직과 구현 패턴을 설명합니다.

Opcode 트레이트

모든 opcode 처리는 `Opcode` 트레이트를 구현합니다:

```
pub trait Opcode: Debug {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error>;
}
```

이 트레이트는 Geth 실행 단계를 받아서 회로 입력에 필요한 `ExecStep` 과 연산들을 생성합니다.

Opcode 디스패칭

`fn_gen_associated_ops` 함수는 `OpcodeId` 에 따라 적절한 처리 함수를 반환합니다:

```
fn fn_gen_associated_ops(opcode_id: &OpcodeId) -> FnGenAssociatedOps {
```

```
if  opcode_id.is_push_with_data() {
return PushN::gen_associated_ops;
}
match  opcode_id {
OpcodeId::ADD => ArithmeticOpcode::<{ OpcodeId::ADD }, 2>::gen_associated_ops,
OpcodeId::SLOAD => Sload::gen_associated_ops,
OpcodeId::SSTORE => Sstore::gen_associated_ops,
OpcodeId::CALL | OpcodeId::CALLCODE => CallOpcode::<7>::gen_associated_ops,
OpcodeId::CREATE => Create::<false>::gen_associated_ops,
// ... 더 많은 opcode들
}
}
```

Opcode 처리 패턴

1. 스택 전용 연산 (Stack-Only Operations)

가장 단순한 형태의 opcode로, 스택에서 값을 읽고 쓰기만 합니다.

구현: `StackPopOnlyOpcode<N_POP, IS_ERR>`

예시: POP, JUMP, 산술 연산 (ADD, MUL, SUB 등)

```
impl<const N_POP: usize, const IS_ERR: bool> Opcode for StackPopOnlyOpcode<N_POP
fn  gen_associated_ops(
state: &mut CircuitInputStateRef,
```

```
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
let mut exec_step = state.new_step(geth_step)?;
// N_POP개의 스택 읽기
let _stack_inputs = state.stack_pops(&mut exec_step, N_POP)?;
if IS_ERR {
// 에러 처리
let next_step = geth_steps.get(1);
let err = state.get_step_err(geth_step, next_step);
exec_step.error = err.unwrap();
state.handle_return((None, None), &mut [&mut exec_step], geth_steps, true)?;
}
Ok(vec![exec_step])
}
}
```

처리 과정:

1. 새로운 `ExecStep` 생성
2. 스택에서 N개의 값 읽기 (`stack_pops`)
3. 에러가 있으면 에러 처리
4. `ExecStep` 반환

2. 메모리 연산 (Memory Operations)

메모리를 읽거나 쓰는 opcode입니다.

예시: MLOAD, MSTORE, MSTORE8

MSTORE 구현:

```
impl<const IS_MSTORE8: bool> Opcode for Mstore<IS_MSTORE8> {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
let mut exec_step = state.new_step(geth_step)?;
// 스택에서 offset과 value 읽기
let offset = state.stack_pop(&mut exec_step)?;
let value = state.stack_pop(&mut exec_step)?;
let offset_u64 = offset.as_u64() as usize;
let shift = offset_u64 % 32;
let slot = offset_u64 - shift;
// 메모리 확장
let minimal_length = offset_u64 + if IS_MSTORE8 { 1 } else { 32 };
state.call_ctx_mut()?.memory.extend_at_least(minimal_length);
// 메모리 쓰기 (왼쪽 워드)
state.memory_write_word(&mut exec_step, slot.into(), left_word)?;
if !IS_MSTORE8 {
```

```
// MSTORE의 경우 오른쪽 워드도 쓰기
state.memory_write_word(&mut exec_step, (slot + 32).into(), right_word)?;
}
Ok(vec![exec_step])
}
}
```

처리 과정:

1. 스택에서 offset과 value 읽기
2. 메모리 확장 (필요한 경우)
3. 메모리에 값 쓰기 (32바이트 정렬된 워드 단위)
4. MSTORE8은 1바이트만, MSTORE는 32바이트 쓰기

3. 스토리지 연산 (Storage Operations)

영구 스토리지를 읽거나 쓰는 opcode입니다.

예시: SLOAD, SSTORE

SLOAD 구현:

```
impl Opcode for Sload {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
```

```
let mut exec_step = state.new_step(geth_step)?;

let call_id = state.call()?.call_id;

let contract_addr = state.call()?.address;

// CallContext 읽기 (TxId, RwCounterEndOfReversion 등)

state.call_context_read(&mut exec_step, call_id,

CallContextField::TxId, Word::from(state.tx_ctx.id()))?;

// 스택에서 key 읽기

let key = state.stack_pop(&mut exec_step)?;

// 스토리지에서 값 읽기

let value = *state.sdb.get_storage(&contract_addr, &key).1;

let (_, committed_value) = state.sdb.get_committed_storage(&contract_addr, &key)

// StorageOp 생성

state.push_op(&mut exec_step, RW::READ,

StorageOp::new(contract_addr, key, value, value,

state.tx_ctx.id(), *committed_value))?;

// 스택에 값 쓰기

state.stack_push(&mut exec_step, value)?;

// Access list 업데이트

let is_warm = state.sdb.check_account_storage_in_access_list(&(contract_addr, k

state.push_op_reversible(&mut exec_step,

TxAccessListAccountStorageOp {

tx_id: state.tx_ctx.id(),

address: contract_addr,
```

```
key,
is_warm: true,
is_warm_prev: is_warm,
})?;
Ok(vec![exec_step])
}
}
```

처리 과정:

1. CallContext 정보 읽기
2. 스택에서 storage key 읽기
3. StateDB에서 스토리지 값 조회
4. `StorageOp` 생성 (READ)
5. 스택에 값 푸시
6. Access list 업데이트 (warm/cold 상태)

SSTORE 구현:

SSTORE는 더 복잡한데, 가스 환불(refund) 계산이 포함됩니다.

```
impl Opcode for Sstore {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
```

```
let mut exec_step = state.new_step(geth_step)?;
let contract_addr = state.call()?.address;
// CallContext 읽기 (IsStatic 체크 포함)
state.call_context_read(&mut exec_step, state.call()?.call_id,
CallContextField::IsStatic,
Word::from(state.call()?.is_static as u8))?;
// 스택에서 key와 value 읽기
let key = state.stack_pop(&mut exec_step)?;
let value = state.stack_pop(&mut exec_step)?;
// 현재 값과 committed 값 조회
let (_, value_prev) = state.sdb.get_storage(&contract_addr, &key);
let (_, committed_value) = state.sdb.get_committed_storage(&contract_addr, &key)
// StorageOp 생성 (WRITE, reversible)
state.push_op_reversible(&mut exec_step,
StorageOp::new(contract_addr, key, value, *value_prev,
state.tx_ctx.id(), *committed_value))?;
// Access list 업데이트
let is_warm = state.sdb.check_account_storage_in_access_list(&(contract_addr, k
state.push_op_reversible(&mut exec_step,
TxAccessListAccountStorageOp { /* ... */ })?;
// 가스 환불 계산 (EIP-3529)
let tx_refund_old = state.sdb.refund();
let tx_refund_new = calc_expected_tx_refund(
```

```
tx_refund_old, value, *value_prev, *committed_value);

state.push_op_reversible(&mut  exec_step,

TxRefundOp {

tx_id: state.tx_ctx.id(),

value: tx_refund_new,

value_prev: tx_refund_old,

})?;

Ok(vec![exec_step])

}

}
```

가스 환불 계산 (`calc_expected_tx_refund`):

- EIP-3529에 따라 SSTORE_CLEARS_SCHEDULE = 4800
- 슬롯 삭제 시 환불 추가
- 슬롯 재생성 시 환불 차감
- 원래 값으로 리셋 시 환불 추가

4. CALL 계열 Opcode

가장 복잡한 opcode 중 하나로, 새로운 실행 컨텍스트를 생성합니다.

종류:

- `CALL` (7개 인자): 다른 계약 호출, value 전송 가능
- `CALLCODE` (7개 인자): 다른 계약 코드를 현재 컨텍스트에서 실행
- `DELEGATECALL` (6개 인자): 다른 계약 코드를 호출자 컨텍스트로 실행

- STATICCALL (6개 인자): 상태 변경 불가능한 호출

구현: CallOpcode<N_ARGS>

```
impl<const N_ARGS: usize> Opcode for CallOpcode<N_ARGS> {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
let mut exec_step = state.new_step(geth_step)?;
// 스택에서 인자 읽기
let [args_offset, args_length, ret_offset, ret_length] = /* ... */;
// 메모리 확장
state.call_expand_memory(args_offset, args_length, ret_offset, ret_length)?;
// Caller 정보 수집
let caller_call = state.call()?.clone();
let caller_balance = /* ... */;
let call_value = /* ... */;
// Precheck: depth < 1025 && balance >= value
let is_precheck_ok = geth_step.depth < 1025 &&
(!is_call_or_callcode || caller_balance >= call_value);
// Callee call 파싱
let callee_call = if is_precheck_ok {
```

```
state.parse_call(geth_step)?

} else {

// 실패 케이스 처리

state.tx_ctx.call_is_success_offset += 1;

let mut call = state.parse_call_partial(geth_step)?;

call.is_success = false;

call

};

// ... (계속)

}

}
```

CALL 처리 분기:

CALL opcode는 4가지 경로로 분기됩니다:

1. **Precompile 호출**: `is_precompile == true`

- Precompile 함수 실행
- 입력 데이터 복사 (caller memory → RLC accumulator)
- 결과 데이터 복사 (RLC accumulator → callee memory)
- 반환 데이터 복사 (callee memory → caller memory)
- OOG 에러 처리 (Modexp 제외)

2. **빈 코드 호출**: `is_empty_code_hash == true`

- 즉시 반환
- return data 비우기

- `handle_return` 호출

3. **일반 계약 호출**: `is_precheck_ok && !is_empty_code_hash`

- Caller context 저장 (PC, SP, GasLeft, MemorySize 등)
- Callee context 초기화
- Value 전송 (CALL의 경우)
- 새로운 call context로 전환

4. **실패 케이스**: `!is_precheck_ok`

- Depth 초과 또는 잔액 부족
- 즉시 반환 (is_success = false)
- `handle_return` 호출

CallContext 필드 관리:

```
// Caller context 저장
for (field, value) in [
(CallContextField::ProgramCounter, (geth_step.pc.0 + 1).into()),
(CallContextField::StackPointer, stack_pointer.into()),
(CallContextField::GasLeft, gas_left.into()),
(CallContextField::MemorySize, next_memory_word_size.into()),
(CallContextField::ReversibleWriteCounter, rwc.into()),
] {
state.call_context_write(&mut  exec_step, caller_call.call_id, field, value)?;
}
```

```
// Callee context 초기화
for (field, value) in [
(CallContextField::CallerId, caller_call.call_id.into()),
(CallContextField::TxId, tx_id.into()),
(CallContextField::Depth, callee_call.depth.into()),
(CallContextField::CallerAddress, callee_call.caller_address.to_word()),
(CallContextField::CalleeAddress, callee_call.address.to_word()),
(CallContextField::Value, callee_call.value),
(CallContextField::IsStatic, (callee_call.is_static as u64).into()),
// ... 더 많은 필드들
] {
state.call_context_write(&mut  exec_step, callee_call.call_id, field, value)?;
}
```

5. CREATE 계열 Opcode

새로운 계약을 생성하는 opcode입니다.

종류:

- `CREATE` : RLP(sender_address, nonce)로 주소 계산
- `CREATE2` : keccak256(0xff, sender_address, salt, keccak256(init_code))로 주소 계산

구현: `Create<IS_CREATE2>`

```
impl<const IS_CREATE2: bool> Opcode for Create<IS_CREATE2> {
fn  gen_associated_ops(
```

```
state: &mut CircuitInputStateRef,

geth_steps: &[GethExecStep],

) -> Result<Vec<ExecStep>, Error> {

let  geth_step = &geth_steps[0];

let  mut  exec_step = state.new_step(geth_step)?;

// 계약 주소 계산

let  address = if IS_CREATE2 {

state.create2_address(&geth_steps[0])?

} else {

state.create_address()?

};

// 주소 충돌 체크

let  callee_account = state.sdb.get_account(&address).1;

let  is_address_collision =

callee_account.code_hash != CodeDB::empty_code_hash() ||

callee_account.nonce > Word::zero();

// Precheck

let  caller_balance = state.sdb.get_balance(&caller.address);

let  caller_nonce = state.sdb.get_nonce(&caller.address);

let  is_precheck_ok =

depth < 1025 &&

caller_balance >= callee_value &&

caller_nonce < u64::MAX;
```

```
// ... (계속)

}

}
```

CREATE 처리 과정:

1. **주소 계산**:

- CREATE: `keccak256(rlp([sender_address, nonce]))`
- CREATE2: `keccak256(0xff ++ sender_address ++ salt ++ keccak256(init_code))`

2. **Precheck**:

- Depth < 1025
- Caller balance >= value
- Caller nonce < u64::MAX
- 주소 충돌 없음

3. **성공 케이스** (`is_precheck_ok && !is_address_collision`):

- Caller nonce 증가
- 계약 주소를 access list에 추가
- Init code를 메모리에서 bytecode로 복사
- Value 전송
- Callee nonce를 1로 설정 (EIP-161)
- Callee context 초기화

4. **실패 케이스**:

- 즉시 반환 (is_success = false)
- 스택에 0 푸시

Init Code 복사:

```
fn handle_copy(
state: &mut CircuitInputStateRef,
step: &mut ExecStep,
call_id: usize,
offset: usize,
length: usize,
) -> Result<(Vec<u8>, H256, H256), Error> {
let rw_counter_start = state.block_ctx.rwc;
let memory = &state.call_ctx()?.memory;
// 메모리에서 init code 읽기
let initcode = memory.0[offset..offset + length].to_vec();
let keccak_code_hash = H256(keccak256(&initcode));
let code_hash = CodeDB::hash(&initcode);
// Bytecode로 변환
let bytes = Bytecode::from(initcode.clone()).code;
// 메모리 워드 읽기
let dst_range = MemoryWordRange::align_range(offset, length);
for chunk_index in 0..dst_range.word_count() {
state.memory_read_word(step, chunk_index.into())?;
}
// CopyEvent 생성
```

```
state.push_copy(step, CopyEvent {
src_type: CopyDataType::Memory,
src_id: NumberOrHash::Number(call_id),
dst_type: CopyDataType::Bytecode,
dst_id: NumberOrHash::Hash(code_hash),
// ...
})?;
Ok((initcode, keccak_code_hash, code_hash))
}
```

Keccak 입력 추가:

CREATE와 CREATE2는 주소 계산을 위해 keccak 해시를 사용하므로, 해시 입력을 기록합니다:

```
// CREATE2의 경우
let  address_preimage = std::iter::once(0xffu8)
.chain(caller.address.to_fixed_bytes())
.chain(salt.to_be_bytes())
.chain(keccak_code_hash.to_fixed_bytes())
.collect::<Vec<_>>();

state.block.sha3_inputs.push(address_preimage);
state.block.sha3_inputs.push(initcode);
```

6. RETURN / REVERT Opcode

실행을 종료하고 caller에게 제어를 반환합니다.

구현: `ReturnRevert`

```
impl Opcode for ReturnRevert {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let step = &steps[0];
let mut exec_step = state.new_step(step)?;
// 스택에서 offset과 length 읽기
let offset = state.stack_pop(&mut exec_step)?;
let length = state.stack_pop(&mut exec_step)?;
// 메모리 확장
if !length.is_zero() {
state.call_ctx_mut()?.memory.extend_at_least(
(offset.low_u64() + length.low_u64()).try_into().unwrap()
);
}
let call = state.call()?.clone();
let offset = offset.low_u64() as usize;
let length = length.as_usize();
// ... (계속)
```

```
}

}
```

RETURN/REVERT 처리 케이스:

Case A: CREATE의 성공적인 RETURN (`call.is_create() && call.is_success && length > 0`)

- 반환 데이터를 계약 코드로 저장
- 메모리 → Bytecode 복사
- CodeHash 업데이트

```
if call.is_create() && call.is_success && length > 0 {
let code_info = handle_create(state, &mut exec_step,
Source { id: call.call_id, offset, length })?;
// CodeHash 업데이트
state.push_op_reversible(&mut exec_step, AccountOp {
address: call.address,
field: AccountField::CodeHash,
value: code_info.hash.to_word(),
value_prev: prev_code_hash,
})?;
}
```

Case B: Root call의 종료 (`call.is_root`)

- IsPersistent 읽기
- 트랜잭션 종료 처리

Case C: 일반 call의 종료 (`!call.is_root`)

- Caller context 복원
- Return data 복사 (callee memory → caller memory)

```
if !call.is_root {
state.handle_restore_context(
(Some(offset.into()), Some(length.into())),
&mut exec_step,
steps,
)?;
}
```

Case D: Return data 저장

- CREATE의 성공적인 RETURN은 return data를 저장하지 않음
- 그 외의 경우 caller의 return_data에 저장

```
let return_data = state.call_ctx()?.memory.0
.get(offset..offset + length)
.unwrap_or_default()
.to_vec();

if call.is_create() && step.op == OpcodeId::RETURN {
state.caller_ctx_mut()?.return_data.clear();
} else {
```

```
state.caller_ctx_mut()?.return_data = return_data;

}
```

에러 처리

Opcode 실행 중 에러가 발생할 수 있습니다. `gen_associated_ops` 함수는 에러를 감지하고 적절히 처리합니다.

에러 감지

```
pub fn gen_associated_ops(

opcode_id: &OpcodeId,

state: &mut CircuitInputStateRef,

geth_steps: &[GethExecStep],

) -> Result<Vec<ExecStep>, Error> {

let geth_step = &geth_steps[0];

let mut exec_step = state.new_step(geth_step)?;

let next_step = if geth_steps.len() > 1 {

Some(&geth_steps[1])

} else {

None

};

// 에러 체크

if let Some(exec_error) = state.get_step_err(geth_step, next_step).unwrap() {
```

```
exec_step.error = Some(exec_error.clone());

// 에러별 처리 함수 호출

if  let Some(fn_gen_error_ops) = fn_gen_error_state_associated_ops(geth_step, &e

let  mut  steps = fn_gen_error_ops(state, geth_steps)?;

steps[0].error = Some(exec_error.clone());

return Ok(steps);

}

// 기본 에러 처리

state.handle_return((None, None), &mut [&mut  exec_step], geth_steps, need_resto

return Ok(vec![exec_step]);

}

// 정상 처리

let  fn_gen_associated_ops = fn_gen_associated_ops(opcode_id);

fn_gen_associated_ops(state, geth_steps)

}
```

에러 타입별 처리

```
fn  fn_gen_error_state_associated_ops(

geth_step: &GethExecStep,

error: &ExecError,

) -> Option<FnGenAssociatedOps> {

match  error {
```

```
ExecError::InvalidJump => Some(InvalidJump::gen_associated_ops),

ExecError::InvalidOpcode => Some(StackPopOnlyOpcode::<0, true>::gen_associated_o

// Depth 에러

ExecError::Depth(DepthError::Call) => match  geth_step.op {

OpcodeId::CALL | OpcodeId::CALLCODE => Some(CallOpcode::<7>::gen_associated_ops)

OpcodeId::DELEGATECALL | OpcodeId::STATICCALL => Some(CallOpcode::<6>::gen_assoc

_ => unreachable!(),

},

// OOG 에러

ExecError::OutOfGas(OogError::Call) => Some(OOGCall::gen_associated_ops),

ExecError::OutOfGas(OogError::MemoryCopy) => Some(OOGMemoryCopy::gen_associated_

ExecError::OutOfGas(OogError::SloadSstore) => Some(OOGSloadSstore::gen_associate

// 잔액 부족

ExecError::InsufficientBalance(InsufficientBalanceError::Call) =>

Some(CallOpcode::<7>::gen_associated_ops),

// 기타 에러들

ExecError::WriteProtection => Some(ErrorWriteProtection::gen_associated_ops),

ExecError::ReturnDataOutOfBounds => Some(ErrorReturnDataOutOfBound::gen_associat

_ => None,

}

}
```

주요 에러 타입

1. **InvalidJump**: 유효하지 않은 JUMP 대상
2. **InvalidOpcode**: 정의되지 않은 opcode
3. **Depth**: Call depth 제한 초과 (1024)
4. **OutOfGas**: 가스 부족
   - `OogError::Call` : CALL 실행 중 가스 부족
   - `OogError::MemoryCopy` : 메모리 복사 중 가스 부족
   - `OogError::SloadSstore` : SLOAD/SSTORE 중 가스 부족
5. **StackOverflow/StackUnderflow**: 스택 오버플로우/언더플로우
6. **InsufficientBalance**: 잔액 부족
7. **WriteProtection**: STATICCALL에서 상태 변경 시도
8. **ReturnDataOutOfBounds**: RETURNDATACOPY에서 범위 초과
9. **ContractAddressCollision**: CREATE/CREATE2에서 주소 충돌

공통 헬퍼 함수

ExecStep 생성

```
impl CircuitInputStateRef {
pub fn new_step(&mut self, geth_step: &GethExecStep) -> Result<ExecStep, Erro
let call_ctx = self.call_ctx()?;
ExecStep::new(
geth_step,
call_ctx,
```

```
self.block_ctx.rwc,
call_ctx.reversible_write_counter,
self.tx_ctx.log_id,
)
}
}
```

스택 연산

```
// 스택에서 N개 값 읽기
pub fn stack_pops(&mut self, step: &mut ExecStep, n: usize) -> Result<Vec<Wor
let mut values = Vec::with_capacity(n);
for _ in 0..n {
values.push(self.stack_pop(step)?);
}
Ok(values)
}

// 스택에서 1개 값 읽기
pub fn stack_pop(&mut self, step: &mut ExecStep) -> Result<Word, Error> {
let call_ctx = self.call_ctx_mut()?;
let value = call_ctx.stack.pop()?;
self.push_op(step, RW::READ, StackOp::new(
```

```
self.call()?.call_id,
call_ctx.stack.stack_pointer(),
value,
))?;
Ok(value)
}

// 스택에 값 쓰기
pub fn stack_push(&mut self, step: &mut ExecStep, value: Word) -> Result<(),
let call_ctx = self.call_ctx_mut()?;
call_ctx.stack.push(value)?;
self.push_op(step, RW::WRITE, StackOp::new(
self.call()?.call_id,
call_ctx.stack.stack_pointer(),
value,
))?;
Ok(())
}
```

메모리 연산

```
// 메모리 워드 읽기
```

```
pub fn memory_read_word(&mut self, step: &mut ExecStep, addr: MemoryAddress)
-> Result<Word, Error> {
let call_ctx = self.call_ctx()?;
let value = call_ctx.memory.read_word(addr);
self.push_op(step, RW::READ, MemoryOp::new(
self.call()?.call_id,
addr,
value,
))?;
Ok(value)
}

// 메모리 워드 쓰기
pub fn memory_write_word(&mut self, step: &mut ExecStep, addr: MemoryAddress,
-> Result<(), Error> {
let call_ctx = self.call_ctx_mut()?;
call_ctx.memory.write_word(addr, value);
self.push_op(step, RW::WRITE, MemoryOp::new(
self.call()?.call_id,
addr,
value,
))?;
Ok(())
```

```
}
```

CallContext 연산

```
// CallContext 읽기
pub fn call_context_read(
&mut self,
step: &mut ExecStep,
call_id: usize,
field: CallContextField,
value: Word,
) -> Result<(), Error> {
self.push_op(step, RW::READ, CallContextOp {
call_id,
field,
value,
})
}

// CallContext 쓰기
pub fn call_context_write(
&mut self,
step: &mut ExecStep,
```

```
call_id: usize,
field: CallContextField,
value: Word,
) -> Result<(), Error> {
self.push_op(step, RW::WRITE, CallContextOp {
call_id,
field,
value,
})
}
```

연산 추가

```
// 일반 연산 추가
pub fn push_op<T: Op>(
&mut self,
step: &mut ExecStep,
rw: RW,
op: T,
) -> Result<(), Error> {
let op_ref = self.block.container.insert(Operation::new(self.block_ctx.rwc, rw,
step.bus_mapping_instance.push(op_ref);
self.block_ctx.rwc.inc_pre();
```

```
Ok(())
}

// Reversible 연산 추가 (revert 시 되돌릴 수 있음)
pub fn push_op_reversible<T: Op>(
&mut self,
step: &mut ExecStep,
op: T,
) -> Result<(), Error> {
self.push_op(step, RW::WRITE, op)?;
self.call_ctx_mut()?.reversible_write_counter += 1;
step.reversible_write_counter_delta += 1;
Ok(())
}
```

Opcode 구현 예시

산술 연산 (ArithmeticOpcode)

```
#[derive(Debug, Copy, Clone)]
pub struct ArithmeticOpcode<const OPCODE_ID: u8, const N_POP: usize>;

impl<const OPCODE_ID: u8, const N_POP: usize> Opcode for ArithmeticOpcode<OPCODE
```

```
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
let mut exec_step = state.new_step(geth_step)?;
// N_POP개의 피연산자 읽기
let _inputs = state.stack_pops(&mut exec_step, N_POP)?;
// 결과 쓰기
let result = geth_steps[1].stack.last()?;
state.stack_push(&mut exec_step, result)?;
Ok(vec![exec_step])
}
}
```

사용 예:

- `ADD: ArithmeticOpcode::<{ OpcodeId::ADD }, 2>`
- `MULMOD: ArithmeticOpcode::<{ OpcodeId::MULMOD }, 3>`
- `ISZERO: ArithmeticOpcode::<{ OpcodeId::ISZERO }, 1>`

DUP 연산

```
#[derive(Debug, Copy, Clone)]
```

```rust
pub struct Dup<const N: usize>;


impl<const N: usize> Opcode for Dup<N> {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
let mut exec_step = state.new_step(geth_step)?;
// N번째 스택 값 읽기
let value = state.call_ctx()?.stack.nth_last(N - 1)?;
// 스택 읽기 연산 추가
let stack_pointer = state.call_ctx()?.stack.stack_pointer();
state.push_op(&mut exec_step, RW::READ, StackOp::new(
state.call()?.call_id,
StackAddress(stack_pointer.0 + N - 1),
value,
))?;
// 복사한 값 푸시
state.stack_push(&mut exec_step, value)?;
Ok(vec![exec_step])
}
```

```
}
```

SWAP 연산

```
#[derive(Debug, Copy, Clone)]
pub struct Swap<const N: usize>;

impl<const N: usize> Opcode for Swap<N> {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
let mut exec_step = state.new_step(geth_step)?;
let call_ctx = state.call_ctx_mut()?;
let stack_pointer = call_ctx.stack.stack_pointer();
// 두 값 읽기
let value_top = call_ctx.stack.nth_last(0)?;
let value_nth = call_ctx.stack.nth_last(N)?;
// 읽기 연산 추가
state.push_op(&mut exec_step, RW::READ, StackOp::new(
state.call()?.call_id,
stack_pointer,
```

```
value_top,
))?;
state.push_op(&mut  exec_step, RW::READ, StackOp::new(
state.call()?.call_id,
StackAddress(stack_pointer.0 + N),
value_nth,
))?;
// 스왑 수행
call_ctx.stack.swap_with_top(N)?;
// 쓰기 연산 추가
state.push_op(&mut  exec_step, RW::WRITE, StackOp::new(
state.call()?.call_id,
stack_pointer,
value_nth,
))?;
state.push_op(&mut  exec_step, RW::WRITE, StackOp::new(
state.call()?.call_id,
StackAddress(stack_pointer.0 + N),
value_top,
))?;
Ok(vec![exec_step])
}
```

```
}
```

LOG 연산

```
#[derive(Debug, Copy, Clone)]
pub struct Log;

impl Opcode for Log {
fn gen_associated_ops(
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
let mut exec_step = state.new_step(geth_step)?;
// LOG0~LOG4에 따라 topic 개수 결정
let topic_count = geth_step.op.postfix().expect("LOG opcode should have postfix
// 스택에서 offset, length 읽기
let offset = state.stack_pop(&mut exec_step)?;
let length = state.stack_pop(&mut exec_step)?;
// Topic들 읽기
let mut topics = Vec::with_capacity(topic_count);
for _ in 0..topic_count {
topics.push(state.stack_pop(&mut exec_step)?);
```

```
}

// 메모리 확장

let  offset_usize = offset.as_usize();

let  length_usize = length.as_usize();

if  length_usize > 0 {

state.call_ctx_mut()?.memory.extend_at_least(offset_usize + length_usize);

}

// LogOp 생성

state.push_op_reversible(&mut  exec_step, LogOp {

address: state.call()?.address,

topics,

data_offset: offset_usize,

data_length: length_usize,

})?;

Ok(vec![exec_step])

}

}
```

특수 케이스

Precompile 호출

Precompile은 EVM에 내장된 특수 계약으로, 특정 주소(0x01~0x09 등)에 위치합니다.

지원되는 Precompile:

- 0x01: ECRecover (ECDSA 서명 복구)
- 0x02: SHA256
- 0x03: RIPEMD160
- 0x04: Identity (데이터 복사)
- 0x05: ModExp (모듈러 지수 연산)
- 0x06: ECAdd (타원곡선 점 덧셈)
- 0x07: ECMul (타원곡선 스칼라 곱셈)
- 0x08: ECPairing (타원곡선 페어링)
- 0x09: Blake2f

Precompile 처리 흐름:

1. **입력 데이터 복사**: Caller memory → RLC accumulator
2. **Precompile 실행**: `execute_precompiled` 함수 호출
3. **결과 저장**: RLC accumulator → Callee memory (주소 0부터)
4. **반환 데이터 복사**: Callee memory → Caller memory
5. **가스 계산**: Precompile별 가스 비용 계산

```
// Precompile 실행
let (result, precompile_call_gas_cost, has_oog_err) = execute_precompiled(
&code_address,
&input_bytes,
callee_gas_left_with_stipend,
);
```

```
// OOG 에러 처리
if has_oog_err && precompile_call != PrecompileCalls::Modexp {
let mut oog_step = ErrorOOGPrecompile::gen_associated_ops(
state, &geth_steps[1], callee_call)?;
oog_step.gas_left = Gas(callee_gas_left_with_stipend);
oog_step.gas_cost = GasCost(precompile_call_gas_cost);
state.handle_return((None, None),
&mut [&mut exec_step, &mut oog_step], geth_steps, true)?;
return Ok(vec![exec_step, oog_step]);
}
```

SELFDESTRUCT

SELFDESTRUCT는 계약을 파괴하고 잔액을 수신자에게 전송합니다.

처리 과정:

1. 수신자 주소를 스택에서 읽기
2. 수신자를 access list에 추가
3. 발신자 잔액을 0으로 설정
4. 발신자 nonce를 0으로 설정
5. 발신자 code hash를 0으로 설정
6. 수신자에게 잔액 전송 (발신자 ≠ 수신자인 경우)
7. Persistent call인 경우 StateDB에서 계약 파괴 표시

```
// 잔액 전송
```

```
if receiver != sender {
state.transfer_to(&mut exec_step, receiver,
!receiver_account.is_empty(), false, value, true)?;
}


// 계약 파괴
if state.call()?.is_persistent {
state.sdb.destruct_account(sender);
}
```

BeginTx / EndTx

트랜잭션의 시작과 끝을 처리하는 특수 단계입니다.

BeginTx:

- 트랜잭션 ID 설정
- Caller nonce 증가
- Caller 잔액에서 가스 비용 차감
- Callee에게 value 전송 (있는 경우)
- Access list 초기화
- Intrinsic gas 계산

EndTx:

- 가스 환불 처리
- 남은 가스를 caller에게 반환
- Coinbase에게 가스 비용 지급

- 트랜잭션 receipt 생성

```
pub fn gen_associated_steps(
state: &mut CircuitInputStateRef,
execution_step: ExecState,
) -> Result<Vec<ExecStep>, Error> {
let fn_gen_associated_steps = match execution_step {
ExecState::BeginTx => gen_begin_tx_steps,
ExecState::EndTx => gen_end_tx_steps_adapt,
_ => unreachable!(),
};
fn_gen_associated_steps(state)
}
```

디버깅 및 검증

메모리 일관성 체크

```
#[cfg(feature = "enable-memory")]
if GETH_TRACE_CHECK_LEVEL.should_check() {
if state.call_ctx()?.memory != geth_steps[0].memory {
log::error!("wrong mem before {:?}", opcode_id);
log::error!("state mem {:?}", &state.call_ctx()?.memory);
log::error!("step mem {:?}", &geth_steps[0].memory);
```

```
if GETH_TRACE_CHECK_LEVEL.should_panic() {
panic!("mem wrong");
}
}
}
```

스택 일관성 체크

```
#[cfg(feature = "enable-stack")]
if GETH_TRACE_CHECK_LEVEL.should_check() {
if  state.call_ctx()?.stack != geth_steps[0].stack {
log::error!("wrong stack before {:?}", opcode_id);
log::error!("state stack {:?}", &state.call_ctx()?.stack);
log::error!("step stack {:?}", &geth_steps[0].stack);
if GETH_TRACE_CHECK_LEVEL.should_panic() {
panic!("stack wrong");
}
}
}
```

가스 검증

```
// CALL의 경우 가스 계산 검증
```

```
if  geth_steps[0].op == OpcodeId::CALL
&& geth_steps[1].depth == geth_steps[0].depth + 1
&& geth_steps[1].gas.0 != callee_gas_left_with_stipend {
let  info = format!("callee_gas_left {} gas_specified {} gas_cost {}",
callee_gas_left, gas_specified, gas_cost);
debug_assert_eq!(geth_steps[1].gas.0, callee_gas_left_with_stipend, "{}", info);
}
```

요약

Bus-Mapping의 opcode 처리는 다음과 같은 계층 구조를 가집니다:

1. **디스패칭 레이어**: `gen_associated_ops` → `fn_gen_associated_ops`

- OpcodeId에 따라 적절한 처리 함수 선택
- 에러 감지 및 에러 처리 함수 호출

2. **Opcode 구현 레이어**: 각 opcode의 `Opcode` 트레이트 구현

- `ExecStep` 생성
- 스택, 메모리, 스토리지 연산 생성
- CallContext 관리
- 특수 로직 처리 (CALL, CREATE, RETURN 등)

3. **헬퍼 함수 레이어**: `CircuitInputStateRef` 의 유틸리티 함수들

- `stack_pop` , `stack_push`
- `memory_read_word` , `memory_write_word`
- `call_context_read` , `call_context_write`

- `push_op`, `push_op_reversible`

4. **상태 관리 레이어**: `StateDB`, `CallContext`, `TransactionContext`

- 계정 상태 관리
- Call stack 관리
- 트랜잭션 컨텍스트 관리

핵심 원칙:

- 모든 상태 변경은 `Operation` 으로 기록
- Reversible 연산은 revert 시 되돌릴 수 있음
- 메모리와 스택은 32바이트 워드 단위로 관리
- 에러는 조기에 감지하고 적절히 처리

관련 문서

- 01-개요.md: Bus-Mapping의 전체 구조
- 02-트레이스-수집.md: Geth 트레이스 수집
- 03-RW-연산.md: RW 연산의 상세 구조
- .../02-핵심-회로/01-EVM-Circuit.md: EVM Circuit에서의 opcode 검증

참고 코드

- `bus-mapping/src/evm/opcodes.rs` : Opcode 디스패칭
- `bus-mapping/src/evm/opcodes/*.rs` : 개별 opcode 구현
- `bus-mapping/src/circuit_input_builder.rs` : CircuitInputBuilder
- `bus-mapping/src/operation.rs` : Operation 정의